

COSC 330 Final Project – Demo Guide

Step-by-step instructions for running every part of the project

New to Linux? Every command you need to type is in a grey code block like this: `command`. Copy it exactly. If something goes wrong, a Troubleshooting tip is nearby.

Before You Start – One-Time Setup

Open a terminal. You can do this by pressing `Ctrl + Alt + T` on most Linux systems, or searching "Terminal" in your application menu.

Then run these two commands to install the tools you need:

```
sudo pacman -S --needed gcc make python-pip wget
```

This installs the C compiler, build tool, Python package manager, and file downloader. It will ask for your password. Type it and press Enter (the cursor won't move — that's normal).

```
pip install boto3 botocore
```

This installs the Python library used to talk to S3/MinIO in Part 4.

Navigate to Your Project Folder

Every command below assumes you are inside the project folder. Run this once at the start:

```
cd ~/Documents/Final_for_SP
```

`cd` means "change directory." The `~` symbol is a shortcut for your home folder. You can always check where you are by typing `pwd` (print working directory).

Part 1 – Build and Run the Shell

Step 1 — Go into the part1 folder

```
cd ~/Documents/Final_for_SP/part1
```

Step 2 — Build the libraries and the shell program

```
make
```

You should see output like:

```
gcc -Wall -Wextra -fPIC -c mylib.c -o mylib.o
ar rcs libmylib.a mylib.o
gcc -shared -o libmylib.so mylib.o
gcc -Wall -Wextra -fPIC -o shell shell.c -L. -lmylib -Wl,-rpath,.
```

`make` reads the Makefile and compiles everything automatically. It builds two library files: `libmylib.a` (static) and `libmylib.so` (shared/dynamic).

Step 3 — Confirm the shell is linked to the shared library

```
ldd ./shell
```

Expected output will include a line like:

```
libmylib.so => ./libmylib.so
```

`ldd` lists which shared libraries a program depends on. This proves your shell uses the `.so` file at runtime, which is exactly what the assignment requires.

Step 4 — Run the shell

```
./shell
```

You will see:

```
===== My Math & Utility Shell =====
1. Add two numbers
2. Subtract two numbers
3. Divide two numbers
4. Reverse a string
5. Fibonacci number
6. Exit
Choose an option:
```

For the demo, show each menu option:

- Type 1 → Enter 10 and 5 → should print **Result: 15**
- Type 2 → Enter 10 and 3 → should print **Result: 7**
- Type 3 → Enter 10 and 4 → should print **Result: 2.5**
- Type 3 → Enter 10 and 0 → should print **Error: division by zero**
- Type 4 → Enter hello → should print **Reversed: olleh**
- Type 5 → Enter 10 → should print **fibonacci(10) = 55**
- Type 6 → exits the program

What to show on camera: The menu appearing, at least 3 different operations working, and the divide-by-zero error handling.

Part 2 – systemd Service

systemd is the service manager built into Linux. It starts programs in the background and keeps them running. Every command here requires **sudo** (administrator access).

Step 1 — Make the daemon script executable

```
chmod +x ~/Documents/Final_for_SP/part2/myshell_daemon.sh
```

chmod +x gives the file permission to be run as a program.

Step 2 — Copy the service file to where systemd looks for services

```
sudo cp ~/Documents/Final_for_SP/part2/myshell.service  
/etc/systemd/system/myshell.service
```

/etc/systemd/system/ is the folder systemd watches for custom service files. **sudo** runs the command as an administrator because only admins can write there.

Step 3 — Tell systemd to reload its list of services

```
sudo systemctl daemon-reload
```

Whenever you add or change a **.service** file, you must run this so systemd notices it.

Step 4 — Start the service

```
sudo systemctl start myshell
```

Step 5 — Check that it's running

```
sudo systemctl status myshell
```

You should see something like:

```
• myshell.service - My COSC330 Shell Service
  Loaded: loaded (/etc/systemd/system/myshell.service; disabled)
  Active: active (running) since ...
  Main PID: 12345 (myshell_daemon.s)
```

The green **active (running)** line is what you want to show on camera.

Step 6 — View the live logs

```
sudo journalctl -u myshell -f
```

-u myshell filters logs to only this service. **-f** follows the log live (like a tail). You'll see a new line every 30 seconds. Press **Ctrl + C** to stop watching.

Step 7 — Enable the service to start on boot

```
sudo systemctl enable myshell
```

Step 8 — Stop and disable when done

```
sudo systemctl stop myshell
sudo systemctl disable myshell
```

What to show on camera: **systemctl status** showing **active (running)**, then **journalctl -f** showing log lines appearing.

Part 3 – TCP File Transfer (Two Terminals)

You will need **two terminal windows open at the same time** for this part. Open a second terminal with **Ctrl + Alt + T** or by right-clicking the desktop.

Step 1 — Build the server and client

In **either** terminal:

```
cd ~/Documents/Final_for_SP/part3
make
```

Step 2 — Create a test file to send

```
echo "Hello from the client! This is file1.txt" > file1.txt
```

`echo` prints text. The `>` sends it into a file instead of the screen.

Step 3 — Start the server (Terminal 1)

In **Terminal 1**, run:

```
cd ~/Documents/Final_for_SP/part3
./server
```

You will see:

```
Server listening on port 9000...
```

The server is now waiting. Leave this terminal open and go to Terminal 2.

Step 4 — Send the file (Terminal 2)

In **Terminal 2**, run:

```
cd ~/Documents/Final_for_SP/part3
./client 127.0.0.1 file1.txt
```

`127.0.0.1` is the "loopback" address — it means "this same computer." On real EC2 instances you would use the server's private IP address instead.

Terminal 2 should print:

```
Connected to 127.0.0.1:9000 — sending 'file1.txt' (42 bytes)
Transfer complete: 42 bytes sent
```

Terminal 1 (server) should print:

```
Connection from 127.0.0.1
Expecting 42 bytes
File saved as 'received_file1.txt' (42 bytes)
```

Step 5 — Confirm the file arrived correctly

```
cat received_file1.txt
```

`cat` prints the contents of a file. It should match what you put in `file1.txt`.

What to show on camera: Both terminals side by side. Start the server, then in the second terminal run the client. Show the server printing the confirmation, then `cat received_file1.txt` showing the file contents.

Part 4 – Local S3 File Transfer with MinIO

MinIO is a free, self-hosted program that behaves exactly like Amazon S3. We run it on your own computer so no AWS account is needed.

Step 1 — Download MinIO

```
wget https://dl.min.io/server/minio/release/linux-amd64/minio -O /tmp/minio
chmod +x /tmp/minio
```

`wget` downloads a file from the internet. `chmod +x` makes it executable.

Step 2 — Create a folder for MinIO to store data

```
mkdir -p ~/minio-data
```

Step 3 — Start the MinIO server (open a NEW terminal for this)

Open a **third terminal** and run:

```
MINIO_ROOT_USER=minioadmin MINIO_ROOT_PASSWORD=minioadmin /tmp/minio server
~/minio-data --console-address ":9001"
```

This starts MinIO on port 9000. Leave this terminal running. You can visit `http://localhost:9001` in your browser to see the MinIO web interface (login: `minioadmin / minioadmin`) — great to show on camera.

Step 4 — Run the transfer script (back in your main terminal)

Open your **main terminal** and run:

```
cd ~/Documents/Final_for_SP/part4
python3 s3_transfer.py
```

You should see:

```
[1] Created 'output.txt':
Timestamp : 2026-05-12T...
Hostname  : your-computer
...

[*] Bucket 'cosc330-bucket' created.
[2] Uploaded 'output.txt' -> s3://cosc330-bucket/output.txt

[3] Contents of s3://cosc330-bucket:
    output.txt (108 bytes, ...)

[4] Downloaded s3://cosc330-bucket/output.txt -> 'downloaded_output.txt'

Done.
```

Step 5 — Verify the downloaded file

```
cat downloaded_output.txt
```

The contents should match `output.txt`.

Step 6 — Show the MinIO web interface (optional but great for demo)

Open a browser and go to: `http://localhost:9001`

- Login: `minioadmin / minioadmin`
- Click on "cosc330-bucket" to see `output.txt` listed there.

What to show on camera: The terminal output showing all 4 steps (create, upload, list, download), plus the MinIO browser UI showing the uploaded file in the bucket.

Troubleshooting

Problem	Fix
<code>make: command not found</code>	Run <code>sudo pacman -S make gcc</code>

Problem	Fix
<code>./shell: error while loading shared libraries</code>	Run from inside <code>part1/</code> — the <code>.so</code> file must be in the same folder
<code>systemctl: command not found</code>	You are not on a systemd system — this won't happen on standard Linux
Permission denied when running <code>systemctl</code>	Add <code>sudo</code> before the command
Connection refused in Part 3	Make sure the server is running in Terminal 1 before running the client
MinIO <code>wget</code> fails	Check your internet connection; or download MinIO from a browser manually
<code>ModuleNotFoundError: No module named 'boto3'</code>	Run <code>pip install boto3 botocore</code>
MinIO error <code>endpoint URL</code>	Make sure you started MinIO in Step 3 of Part 4 before running the script

Quick Reference – All Commands in Order

```
# PART 1
cd ~/Documents/Final_for_SP/part1
make
ldd ./shell
./shell

# PART 2
chmod +x ~/Documents/Final_for_SP/part2/myshell_daemon.sh
sudo cp ~/Documents/Final_for_SP/part2/myshell.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl start myshell
sudo systemctl status myshell
sudo journalctl -u myshell -f
sudo systemctl stop myshell

# PART 3 (needs 2 terminals)
cd ~/Documents/Final_for_SP/part3
make
echo "Hello from the client!" > file1.txt
# Terminal 1:
./server
# Terminal 2:
./client 127.0.0.1 file1.txt
cat received_file1.txt

# PART 4 (needs MinIO running in a separate terminal)
# Terminal 3 (MinIO):
```



```
MINIO_ROOT_USER=minioadmin MINIO_ROOT_PASSWORD=minioadmin /tmp/minio server  
~/minio-data --console-address ":9001"  
# Main terminal:  
cd ~/Documents/Final_for_SP/part4  
python3 s3_transfer.py  
cat downloaded_output.txt
```